

Documentação técnica · arquitetura, scripts, física, interface, configurações e correções

Parkour nas Alturas · Jogos Digitais · Fase 2 · Unity 6000.3.6f1 (URP) · Outubro de 2026

VISÃO TÉCNICA

Tecnologias e números do projeto

2
cenas no jogo (Menu e Obby)

19
scripts C# do jogo (≈1.000 linhas)

517
objetos na cena Obby

24
materiais · 9 sons · 7 imagens de interface

Pilha de tecnologia

Item	Valor
Motor	Unity 6000.3.6f1 (versão usada pelo grupo)
Renderização	URP 17.3 (Universal Render Pipeline), perfil "PC"
Interface	UGUI 2.0 (Canvas, Text, Image, Button)
Entrada	Input Manager clássico (`Input.GetAxis`), com *Active Input Handling = Both*
Física	PhysX: CharacterController, BoxCollider, MeshCollider e triggers
Persistência	PlayerPrefs (recorde, estrelas e opções)
Linguagem	C# (MonoBehaviour e uma classe estática)
Versionamento	Git/GitHub, branch `parkour-nas-alturas`

Fluxo entre cenas

Menu Jogar → **Obby** chegada → **Vitória**

Obby Esc → **Pausa** → Continuar / Opções / Reiniciar / Menu

Vitória → Jogar de novo (Obby) / Menu

Toda troca de cena passa pelo `Transicao.IrPara(cena)`, que escurece a tela (fade), carrega a cena nova com `SceneManager.LoadScene` e clareia de novo.

A cena **Obby** tem um único "cérebro", o **HUD** (gerenciador da partida), que os outros scripts acessam por `HUD.instancia` (padrão *singleton* simples).

ORGANIZAÇÃO

Pastas e hierarquia da cena

Pastas (Assets)

```
Assets/
├─ Scenes/
│  ├─ Menu.unity          tela inicial
│  ├─ Obby.unity          as 7 fases
│  └─ SampleScene.unity   cena original do grupo
├─ Zona3/
│  ├─ *.cs                19 scripts do jogo
│  ├─ Materiais/          24 materiais (Obby_*, lava, vidro…)
│  ├─ Sons/               música e efeitos .wav
│  └─ UI/                 ícones, vinheta, fonte Poppins
├─ Settings/              URP (qualidade, pós-processamento)
└─ TorusGenerator.cs      gerador dos anéis (grupo)
```

Hierarquia da cena Obby

```
Obby (cena)
├─ Main Camera            CameraSegue
├─ Directional Light      sol + sombras
├─ Global Volume          bloom e ajuste de cor
├─ Fundo_Agua             plano da água
├─ Player                 CharacterController, Personagem
│  └─ Modelo              boneco de blocos (AnimacaoBoneco)
├─ Obby
│  ├─ Fase 1 - Blocos     Checkpoint, Blocos, Lava (Perigo)
│  ├─ Fase 2 - Curva      Tábuas
│  ├─ Fase 3 - Bolinhas   Discos
│  ├─ Fase 4 - Barra…     Postes, Girar, Trampolim
│  ├─ Fase 5 - Aneis      TorusGenerator
│  ├─ Fase 6 - …moveis    PlataformaMovel
│  ├─ Fase 7 - Ponte…     Vidros, Chegada
│  └─ Nuvens
├─ HUD (Canvas)           HUD.cs, painéis, Transicao
└─ EventSystem
```

JOGADOR

Movimento do personagem

```
// Personagem.cs (resumo)
public float velocidade = 7f;
public float alturaPulo = 2.2f;
public float gravidade  = -25f;
public float ajudaNoPulo = 0.15f; // "coyote time"

void Update() {
  // 1. direção relativa à câmera
  Vector3 frente = cameraJogo.forward; frente.y = 0;
  Vector3 direita = cameraJogo.right;  direita.y = 0;
  Vector3 dir = frente.normalized * z + direita.normalized * x;

  // 2. pulo com folga de 0,15 s
  if (tempoFora < ajudaNoPulo && tempoBotao < ajudaNoPulo)
    velocidadeY = Mathf.Sqrt(alturaPulo * -2f * gravidade);

  // 3. gravidade e 4. um único Move por quadro
  velocidadeY += gravidade * Time.deltaTime;
  Vector3 mov = dir * velocidade; mov.y = velocidadeY;
  controller.Move(mov * Time.deltaTime);
}
```

Como funciona

- **CharacterController** (altura 2, raio 0,5, degrau 0,3): faz a colisão sem usar Rigidbody, o que deixa o controle mais preciso para parkour.
- **Andar relativo à câmera:** W sempre vai "para frente" do que a câmera mostra, como no Roblox.
- **Força do pulo:** calculada pela fórmula da física `v = √(2·g·h)`. Com h = 2,2 m e g = 25 m/s², a velocidade inicial é ≈ 10,5 m/s.
- **Coyote time:** o pulo ainda vale até 0,15 s depois de sair da beirada, e o botão apertado um pouco antes de cair também conta. Isso deixa o jogo menos "injusto".
- **Chão:** quando está no chão, a velocidade vertical fica em -2 para o boneco não "flutuar" nas descidas.
- **Visual:** o filho *Modelo* gira suavemente (`Quaternion.Slerp`) para onde o jogador anda, e o **AnimacaoBoneco** balança braços e pernas com um seno quando a velocidade passa de 0,5.

CÂMERA

Câmera em terceira pessoa

```
// CameraSegue.cs (resumo) — roda no LateUpdate
if (Input.GetMouseButton(1)) {   // botão direito
  anguloHorizontal += Input.GetAxis("Mouse X") * sens;
  anguloVertical   -= Input.GetAxis("Mouse Y") * sens;
}
anguloVertical = Mathf.Clamp(anguloVertical, 5f, 75f);
distancia = Mathf.Clamp(distancia, 4f, 16f);  // zoom

Quaternion rot = Quaternion.Euler(anguloVertical, anguloHorizontal, 0);
Vector3 foco = alvo.position + Vector3.up;
Vector3 direcao = -(rot * Vector3.forward);

// não atravessar plataformas
RaycastHit[] hits = Physics.SphereCastAll(foco, 0.3f,
                      direcao, distancia, ~0,
                      QueryTriggerInteraction.Ignore);
// ... pega o menor hit e aproxima a câmera

transform.position = foco + direcao * distanciaAtual;
transform.LookAt(foco);
```

Detalhes

- **Coordenadas esféricas:** dois ângulos (horizontal e vertical) e uma distância definem onde a câmera fica em volta do jogador.
- **LateUpdate:** a câmera se move depois do personagem no mesmo quadro, então não "treme".
- **SphereCast:** uma esfera de raio 0,3 é lançada do jogador até a câmera. Se bater numa plataforma, a câmera chega mais perto, nunca a menos de 1,5 m.
- **Suavização assimétrica:** a câmera se aproxima rápido (para não entrar na parede) e volta devagar, usando `1 − e^(−v·dt)`, que funciona igual em qualquer FPS.
- **Sensibilidade** vem das Opções (`Configuracoes.Sensibilidade` , de 0,3 a 2). Q/E giram pelo teclado.
- Usa `Time.unscaledDeltaTime` , então dá para girar a câmera até com o jogo pausado.

REGRAS DO JOGO

Gerenciador da partida (HUD.cs)

```
public static HUD instancia;   // acesso global

public bool Checkpoint(int fase, Vector3 pos, string msg) {
  if (fase <= faseAtual) return false;  // não volta fase
  faseAtual = fase; pontoDeVolta = pos;
  AtualizarBarra(); MostrarAviso(msg); TocarSom(somCheckpoint);
  return true;
}

public void Morrer() {
  mortes++; TocarSom(somMorte);
  flashMorte.color = new Color(1f, .1f, .15f, .55f);
  player.SetParent(null);          // sai da plataforma móvel
  cc.enabled = false;               // teleporte seguro
  player.position = pontoDeVolta;
  cc.enabled = true;
}

// estrelas na vitória
estrelas = 1;
if (moedas >= Mathf.CeilToInt(total * 0.75f)) estrelas++;
if (mortes <= 3) estrelas++;
```

Responsabilidades

- Conta **moedas**, **tempo** e **mortes** e atualiza o HUD a cada quadro.
- Guarda o **ponto de volta** do último checkpoint. Morre quem cai abaixo de `y = −1` ou encosta num **Perigo**.
- O **CharacterController** é desligado durante o teleporte. Sem isso, ele “desfaz” a mudança de posição.
- **Pausa:** `Time.timeScale = 0` congela física e animações, e a interface continua animada porque usa tempo real.
- **Vitória:** calcula as estrelas, salva o recorde e as estrelas e mostra o painel com animação.

Dados salvos (PlayerPrefs)

Chave	Conteúdo	Padrão
`recorde`	melhor tempo (s)	—
`estrelas`	mais estrelas obtidas	0
`opc_musica`	volume da música	0,7
`opc_efeitos`	volume dos efeitos	0,8
`opc_sensibilidade`	sensibilidade da câmera	1,0

FASES

Mecânicas das fases

Script	Tipo de colisor	O que faz
Checkpoint	Trigger	Avisa o HUD e pinta a marca do chão de dourado
Perigo	Trigger	Chama `Morrer()` (lava e barra giratória)
Trampolim	Trigger	Chama `Personagem.Impulso(alturaDoPulo)`
Girar	—	Gira o objeto (graus/s por eixo)
PlataformaMovel	Trigger + sólido	Vai e volta e leva o jogador junto
Moeda	Trigger	Soma no total no `Start` e some ao ser pega
Chegada	Trigger	Chama `Vencer()`
TorusGenerator	MeshCollider	Gera a malha e o colisor dos anéis (raio 0,9 / tubo 0,45)

Ponte de vidro: o vidro verdadeiro tem BoxCollider sólido. O falso tem o mesmo visual, mas o colisor é *Is Trigger*, então o jogador atravessa, cai abaixo de y = -1 e volta ao checkpoint. A resposta não aparece em lugar nenhum do jogo.

```
// PlataformaMovel.cs
void Update() {
  // PingPong: vai de 0 a 1 e volta, sem parar
  float t = Mathf.PingPong(Time.time * velocidade, 1f);
  transform.position = inicio + deslocamento * t;
}
void OnTriggerEnter(Collider outro) {
  if (outro.CompareTag("Player"))
    outro.transform.SetParent(transform);  // anda junto
}
void OnTriggerExit(Collider outro) {
  if (outro.CompareTag("Player") &&
      outro.transform.parent == transform)
    outro.transform.SetParent(null);
}

// Moeda.cs
void Start()  { HUD.instancia.total++; }
void Update() { transform.Rotate(0, velocidadeGiro * Time.deltaTime, 0); }
void OnTriggerEnter(Collider outro) {
  if (outro.CompareTag("Player")) {
    HUD.instancia.PegouMoeda(); Destroy(gameObject);
  }
}
```

Como cada moeda se soma sozinha ao total, dá para duplicar moedas na cena sem mexer em código. Hoje são 14.

INTERFACE

Interface, transições e opções

- **Canvas Scaler:** *Scale With Screen Size*, resolução de referência 1920×1080 e *match* 0,5. A interface fica proporcional em qualquer monitor.
- **Fonte:** Poppins Bold (licença SIL OFL), com Outline e Shadow nos títulos.
- **Animações sem Animator:** tudo é feito por código (**BotaoAnimado**, **Flutuar**, estrelas da vitória) usando `Time.unscaledDeltaTime`, que continua rodando com o jogo pausado.
- **BotaoAnimado:** implementa `IPointerEnterHandler` e `IPointerDownHandler`, cresce 7% no hover, afunda 6% no clique e usa um único AudioSource compartilhado (`static`) para os sons.
- **Opções:** a classe estática **Configuracoes** encapsula o PlayerPrefs com propriedades e `Mathf.Clamp`, então nenhum valor sai do intervalo permitido.
- **Esc centralizado:** se as Opções estiverem abertas, o Esc fecha as Opções. Senão, pausa ou despausa. Isso evita dois scripts reagindo à mesma tecla.

```
// Transicao.cs — fade entre cenas
public static void IrPara(string cena) {
  Time.timeScale = 1f;
  if (instancia == null) { SceneManager.LoadScene(cena); return; }
  instancia.StartCoroutine(instancia.Fade(0f, 1f, cena));
}
IEnumerator Fade(float de, float para, string cena) {
  float t = 0f;
  while (t < duracao) {
    t += Time.unscaledDeltaTime;
    grupo.alpha = Mathf.Lerp(de, para,
                  Mathf.SmoothStep(0f, 1f, t / duracao));
    yield return null;
  }
  if (cena != null) SceneManager.LoadScene(cena);
}

// Configuracoes.cs
public static float Musica {
  get { return PlayerPrefs.GetFloat("opc_musica", 0.7f); }
  set { PlayerPrefs.SetFloat("opc_musica", Mathf.Clamp01(value)); }
}
```

PROJETO

Configurações, versão do Unity e Git

Configurações alteradas

Onde	O quê	Por quê
Player	Active Input Handling = Both	o código usa `Input.GetAxis`
PC_RPAsset (URP)	MSAA 4x	bordas sem serrilhado
PC_RPAsset (URP)	sombra 4096, distância 70	sombras nítidas no mapa todo
Quality (PC)	V-Sync ligado	sem "rasgar" a imagem
Volume	Bloom 0,35 + Color Adjustments	cores mais vivas
Build Profiles	Menu, Obby (SampleScene desligada)	ordem do jogo

Git

- `.gitignore` de Unity: Library, Temp, Logs, UserSettings e .csproj ficam fora do repositório.
- O trabalho vai na branch `parkour-nas-alturas`, criada a partir da `main` atualizada, sem mexer na branch principal.
- O arquivo `ProjectVersion.txt` não entra nos commits, para não forçar outra versão do Unity no grupo.

Mudança para a versão do grupo (6000.6 → 6000.3)

O jogo foi feito no Unity 6000.6, mas o grupo padronizou o projeto no **6000.3.6f1**, na pasta `fase 2 (versão 6.3.1f)`. Uma versão mais antiga do Unity nem sempre lê arquivos salvos numa mais nova, então o trabalho foi migrado:

- As cenas Menu e Obby, os scripts, os materiais e os sons foram copiados para a pasta nova, mantendo os mesmos GUIDs (os arquivos `.meta`), para nenhuma referência quebrar.
- O formato da **Light** nas cenas foi convertido: `serializedVersion 13 → 12` e o campo `m_ShapeRadius → m_ShadowRadius`, igual ao que o 6000.3 grava.
- As configurações (input, qualidade, URP, Build Profiles) foram aplicadas de novo nos arquivos da versão nova, em vez de copiar os antigos.
- O jogo foi aberto e jogado do começo ao fim no 6000.3.6f1 para conferir.

PROBLEMAS E SOLUÇÕES

Erros técnicos e correções

Erro	Causa	Correção
Logo do menu não aparecia	O **Flutuar** salvava `transform.localPosition` no `Start`, antes de o layout da interface calcular a posição pelas âncoras, e reaplicava esse valor errado a cada quadro.	Para elementos de interface, usar `RectTransform.anchoredPosition`, que vem certo da cena.
Ícones do HUD brancos	Os `.meta` das imagens não tinham o tipo de importação, então o Unity importou como textura comum. Os componentes *Image* pediam um *Sprite* que não existia.	*Texture Type = Sprite (2D and UI)*, *Alpha Is Transparency* e *Wrap Clamp* nas 7 imagens.
"Como jogar" encavalado	As dicas tinham 3 linhas com *Vertical Overflow*, o texto crescia para cima sobre a lista de teclas, e o título (pivô no topo) encostava na primeira linha.	Cartão de 780 para 900 de altura, linhas reposicionadas (-205 a -493) e caixa de dicas maior.
Interface vazia (início do projeto)	Na geração das telas, os componentes de interface ficam na camada 5 (UI), mas eram procurados na camada 0, então não eram ligados aos objetos.	Busca corrigida e uma verificação automática de "componente sem dono" antes de salvar a cena.
Zona 3 embaixo do chão	O desenho do mapa foi lido como vista de lado, em vez de vista de cima.	A zona foi refeita como mapa visto de cima, com as alturas corretas.
Disco/anel com colisão errada	O cilindro padrão do Unity usa colisor de cápsula, e o boneco "escorregava" na borda arredondada.	Troca por MeshCollider convexo. O buraco do anel ficou menor que o boneco.
Esc fazendo duas coisas	As Opções e a Pausa reagiam ao Esc ao mesmo tempo.	Tratamento do Esc centralizado no HUD e no Menu.
Commit no caminho antigo	A `main` tinha mudado o projeto de pasta enquanto o trabalho era feito, e a branch estava baseada na versão antiga.	Branch atualizada com a `main` nova e arquivos levados para `fase 2 (versão 6.3.1f)` antes do commit.